

# 8. Intent Inference + Multimodal Visual Understanding + SLAM Navigation + Visual Functionality

## 8. Intent Inference + Multimodal Visual Understanding + SLAM Navigation + Visual Functionality

Introduction

Operation Steps

Preparatory Steps

Getting Started

## Introduction

In this tutorial, we will learn how to issue commands via chat with Muto in the Dify interface.

## Operation Steps

### Preparatory Steps

This lesson requires that the Dify terminal be launched beforehand, and all configuration items have been configured as in the previous section.

The image shows a terminal window with a script for configuring the voice module and Dify. The script includes sections for voice module configuration, Dify configuration, and fallback settings. A red box highlights the line `export FORCE_VOICE_PROVIDER="xunfei"` in the terminal. Below the terminal, the Dify interface is shown with a red box highlighting the 'MutoLLM2.0 EN' plugin card.

```
5
7 # 1. Voice Module Configuration
8 # =====
9
10 # 1.1 Provider & Keys
11 # -----
12 export ALIBABA_API_KEY="sk- [REDACTED]"
13 export FORCE_VOICE_PROVIDER="xunfei"
14
15 # 1.2 General Voice Settings
16 export VOICE_LANGUAGE="en"
17
18 # 2. Dify Configuration
19 # Auto-detect IP from wlan0
20 export DIFY_HOST=$(ip addr show wlan0 2>/dev/null | grep "inet " | awk '{print $2}' | cut -d/ -f1 | head -n 1)
21 # Fallback if wlan0 has no IP
22 if [ -z "$DIFY_HOST" ]; then
23     export DIFY_HOST="127.0.0.1"
24 fi
25
26 if [ "$VOICE_LANGUAGE" = "zh_cn" ]; then
27     export DIFY_API_KEY="app-nlbHG1j5hn8ya68M9VjGcnIh"
28 elif [ "$VOICE_LANGUAGE" = "en" ]; then
29     export DIFY_API_KEY="app-Kw3QqjIXE6xtwjsGVxuvIZU8"
30 fi
```

Dify yaboom@163.com's ...

Explore Studio Knowledge Tools

Created by me All Tags Search

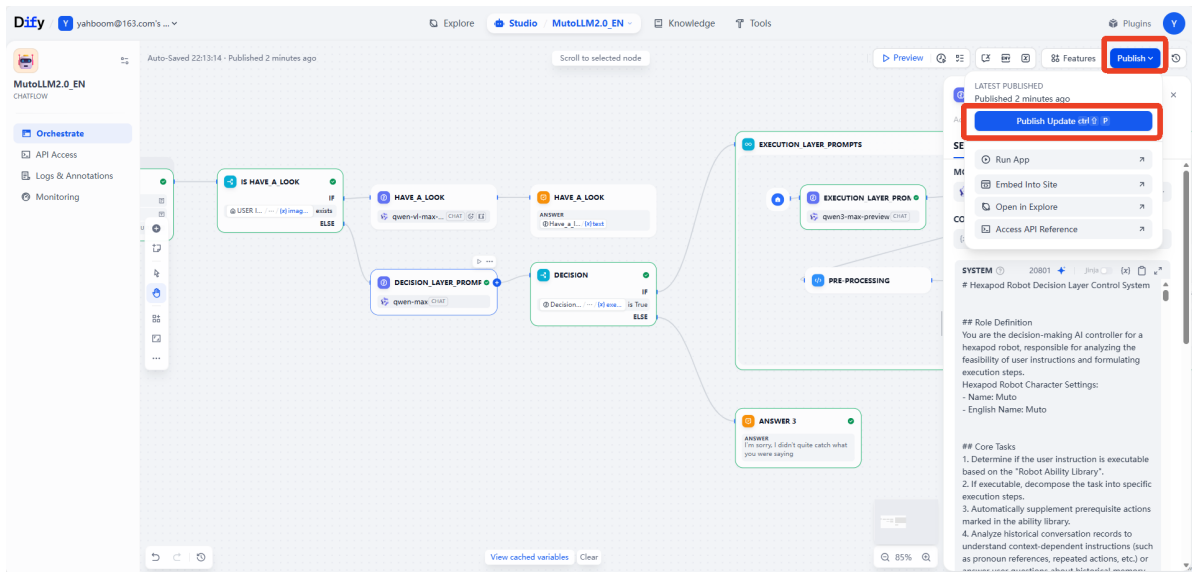
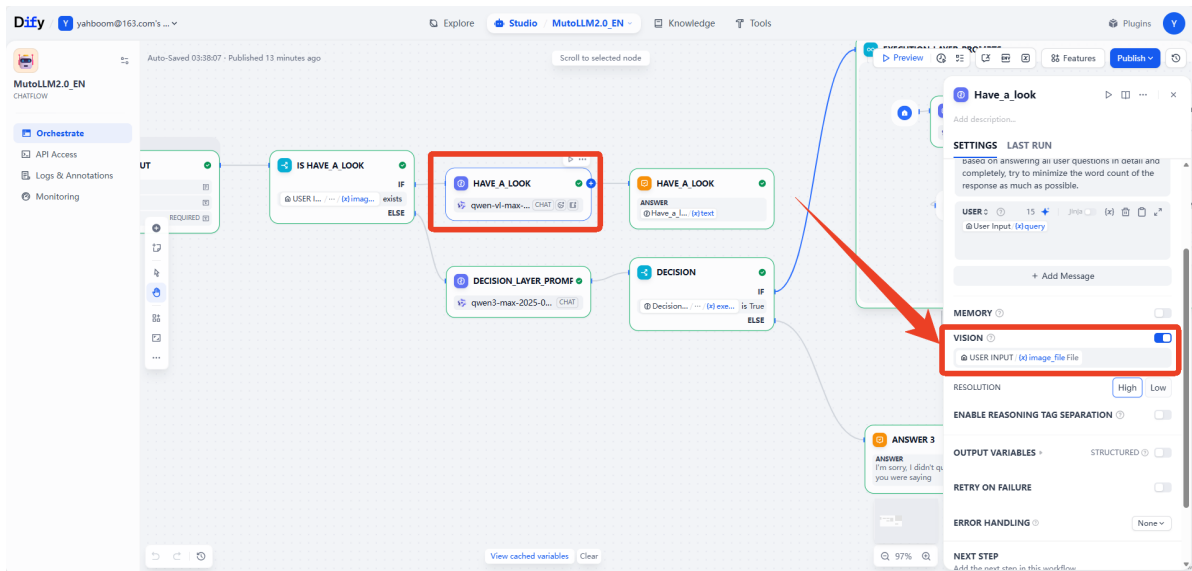
CREATE APP

- Create from Blank
- Create from Template
- Import DSL file

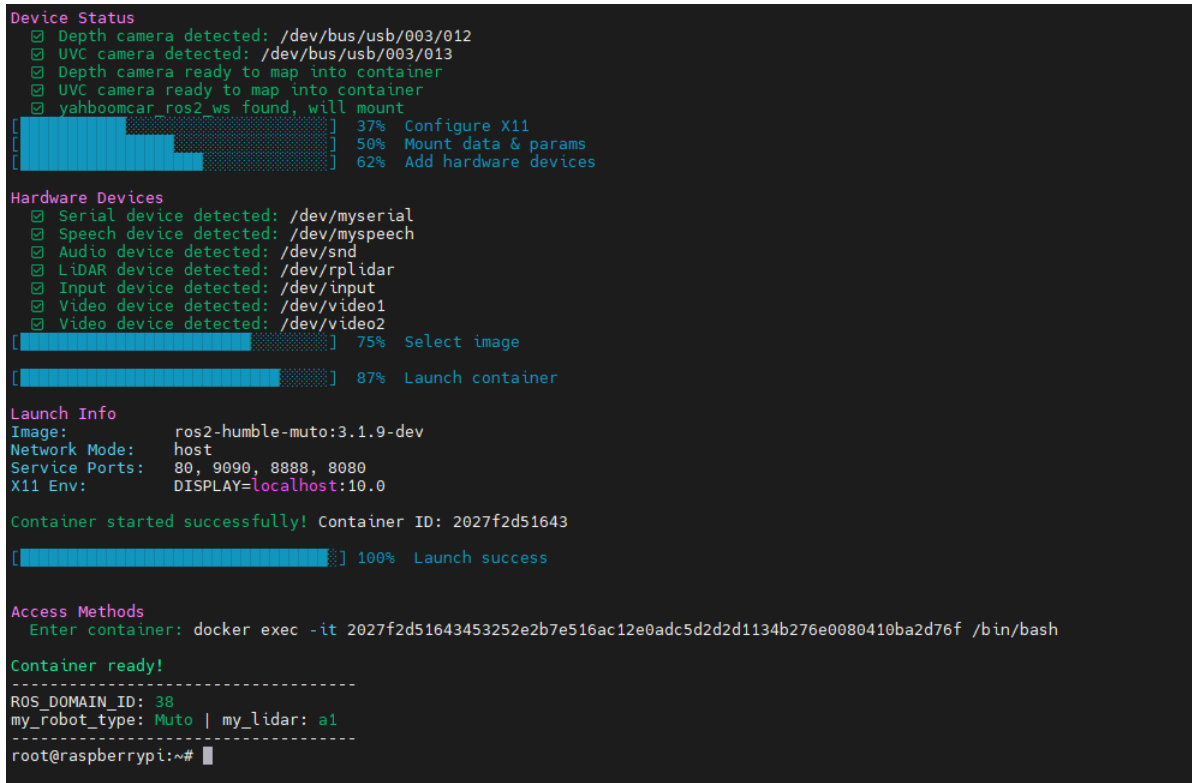
MutoLLM2.0 EN yaboom@163.com · Edited at 01/20/2025 4:30

test yaboom@163.com · Edited at 12/23/2025 11:...

MutoLLM2.0 yaboom@163.com · Edited at 01/12/2025 4:14



This lesson requires MutoRS to be powered on. Following the steps in the previous chapters, start the large model terminal in Docker.



```
cd muto-llm-2.0/ && ./rebuild_and_launch.sh --no-voice
```

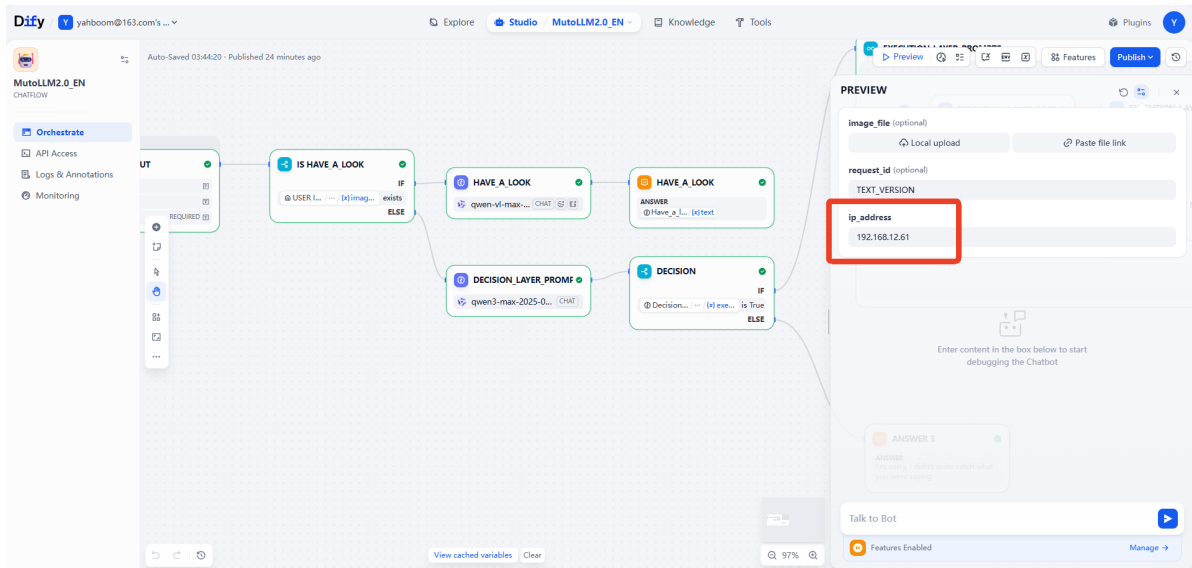
```

[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.voice_interface - INFO - Speech to text initialized successfully with alibaba provider
[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.core.text_to_speech - INFO - TTS Provider alibaba
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - Audio queue manager initialized successfully
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - All voice modules initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315363643] [muto_controller_node]: Command executor initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315920470] [muto_controller_node]: Attempting to prestart camera system...
[muto_controller_node-1] X Node not running: /camera/camera
[muto_controller_node-1] Current running nodes: ['/muto_controller_node', '/navigation_manager']
[muto_controller_node-1] [INFO] [1766137697.078459054] [muto_controller_node]: Camera service started successfully: Camera started successfully
[muto_controller_node-1] [INFO] [1766137697.085229984] [persistent_image_capture_3188dc8d]: Persistent image capture node initialized for topic: /camera/color/image_raw with Best Effort QoS
[muto_controller_node-1] [INFO] [1766137697.086642747] [muto_controller_node]: Persistent camera node initialized: Camera node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.087635385] [muto_controller_node]: FastAPI application created successfully
[muto_controller_node-1] [INFO] [1766137697.149655757] [muto_controller_node]: API routes registered successfully
[muto_controller_node-1] 2025-12-19 09:48:17,149 - voice_module.core.voice_wake - INFO - Starting voice wake detector on port: /dev/myspeech
[muto_controller_node-1] 2025-12-19 09:48:17,154 - utils.serial_comm - INFO - Serial data receiving started
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.core.voice_wake - INFO - Voice wake detector started successfully
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.voice_interface - INFO - Wake detection started
[muto_controller_node-1] [INFO] [1766137697.155236161] [muto_controller_node]: Voice assistant auto-start: True
[muto_controller_node-1] [INFO] [1766137697.156658443] [muto_controller_node]: ROS2 publisher created successfully
[muto_controller_node-1] [INFO] [1766137697.157379009] [muto_controller_node]: Status timer and health check created successfully
[muto_controller_node-1] [INFO] [1766137697.157921559] [muto_controller_node]: Muto Controller Node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.158425850] [muto_controller_node]: Starting FastAPI server on 0.0.0.0:8080
[muto_controller_node-1] [INFO] [1766137697.161239210] [muto_controller_node]: FastAPI server started successfully
[muto_controller_node-1] INFO: Started server process [221]
[muto_controller_node-1] INFO: Waiting for application startup.
[muto_controller_node-1] INFO: Application startup complete.
[muto_controller_node-1] INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)

```

## Getting Started

The large text model requires you to enter Muto's IP address here, which is the address used to access the Dify page.



In this section, based on SLAM navigation, we will add pre-defined **Intent Mapping** to allow Muto to execute behaviors and actions that match your intentions. In this tutorial, we will learn how to add/modify prompts for large models in Dify.

The intent mapping file we use in this section is as follows:

### Intent Mapping

- Instruction 1: "I'm currently in place1, and I'm a little thirsty."
- Intent Analysis 1: Find out which area on the map has bottled water and go to that area to find it.
- Example: 1. Navigate to area A, 2. Check if bottled water is available, 3. If bottled water is found, measure the distance between you and the user, 4. Navigate to the user's current location (place1) and announce the result.

- Instruction 2: "I want to sleep."
- Intent Analysis 2: 1. Navigate to the master bedroom and check if the environment is suitable for sleeping. 2. Tell the user a bedtime story to soothe their emotions.

## Item Storage Locations

### Master Bedroom

- Toilet paper, pen, phone

### Kitchen

- Bottled water, Coke, Sprite, bread

### Zoo

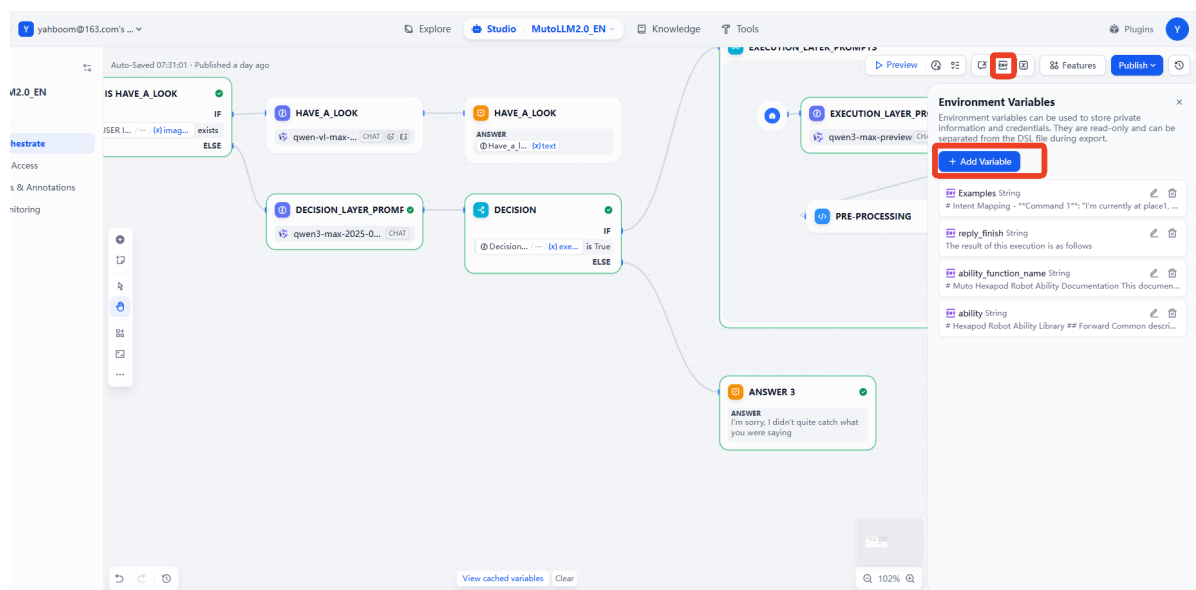
- Monkeys, tigers, peacocks

You can add intent mapping content later based on the actual scenario.

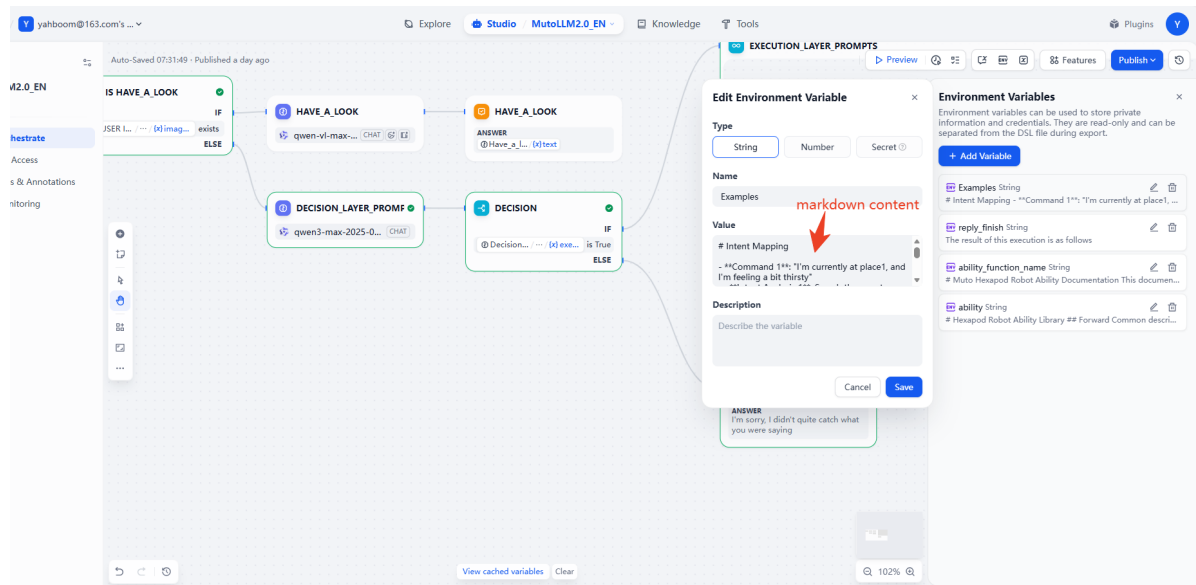
Open the Dify interface, find the decision layer node, and click the ENV button in the upper right corner.

Note: This step is already configured in the factory image. You can directly edit the Examples environment variable to modify the intended meaning.

Click the "Add Variable" button.



Add a variable named `Examples`, of type `String`. Enter the content from the Markdown file into the value.



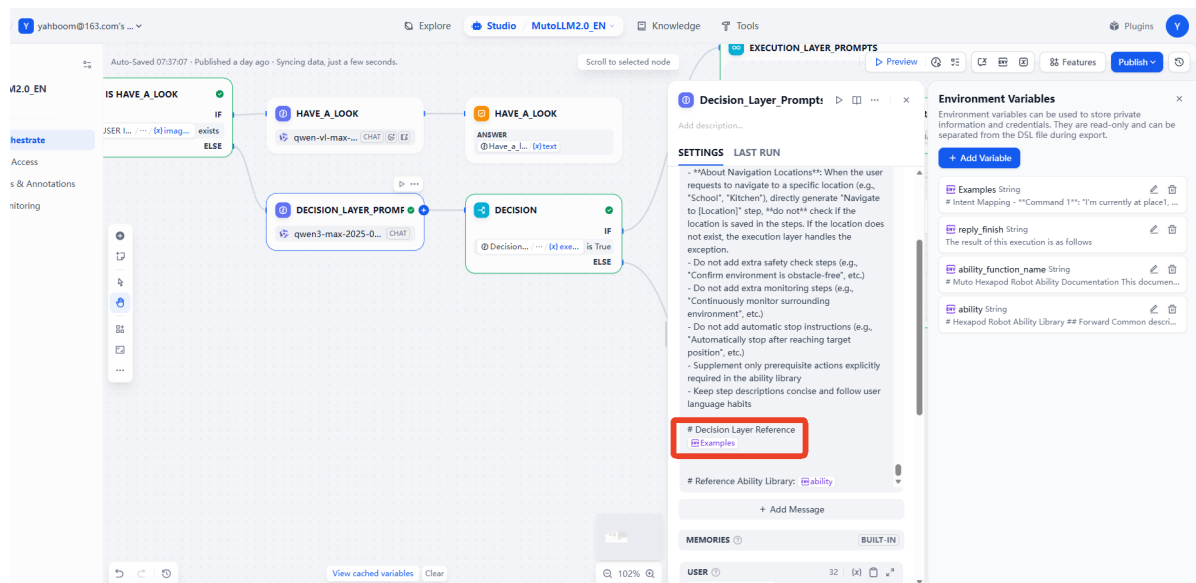
After filling it in, click the Save button in the lower right corner.

Next, return to the System code block in the "Decision Layer Prompts" node.

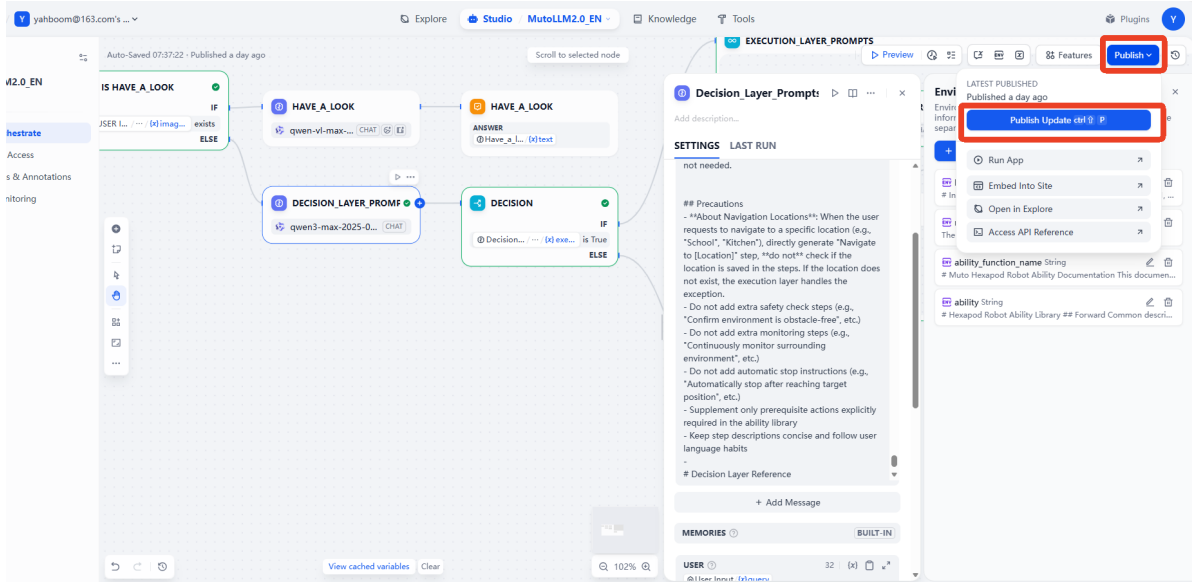
Add the following content at the end: "# Decision Layer Reference"

Then type the forward slash "/" on your keyboard. Select the Example variables we just saved.

The result is shown in the image below.

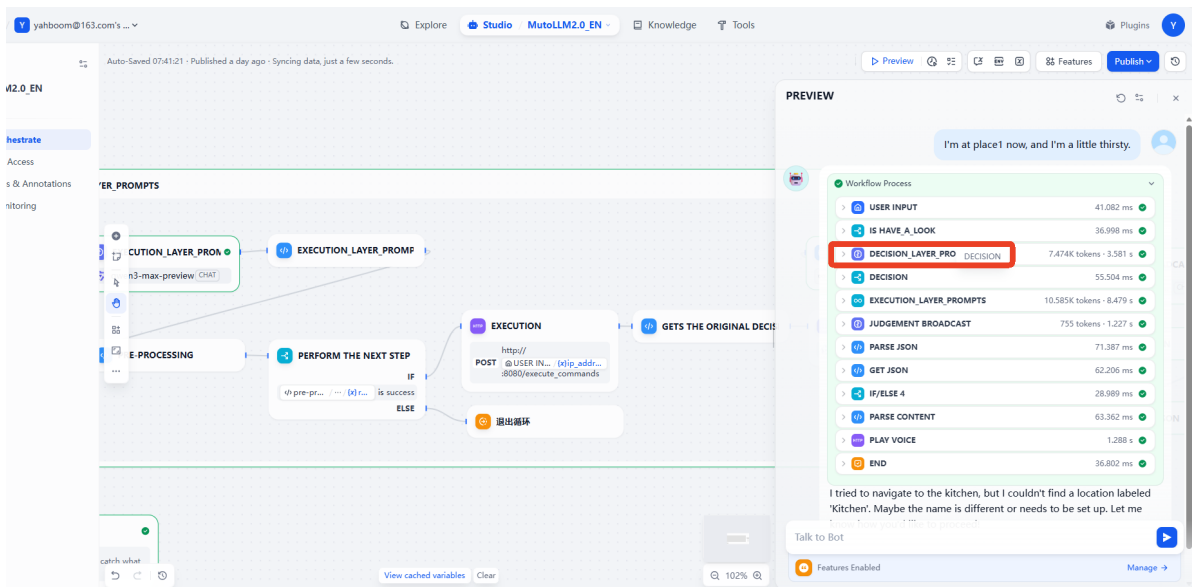


After adding, click the "Publish" button in the upper right corner, then click "Publish Update".

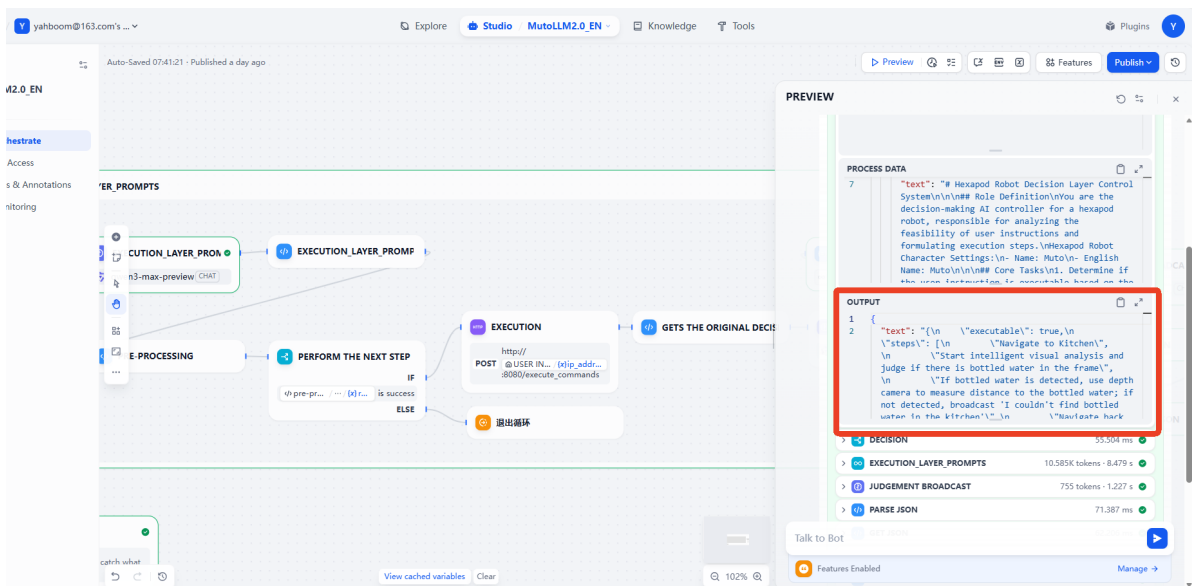


After publishing, we can try out the intent we just added.

We will focus on observing the output results of the decision-making layer.



We can see the output results of the decision-making layer. In the result, the large model has already planned according to the prompts we added.



Here we mainly teach you how to add your own custom prompts, so we haven't demonstrated the complete effect.

If we want to achieve the full functionality, in addition to modifying the prompts, we also need to mark the actual locations of places like "kitchen" and "master bedroom" in advance, and start our navigation system in the background.